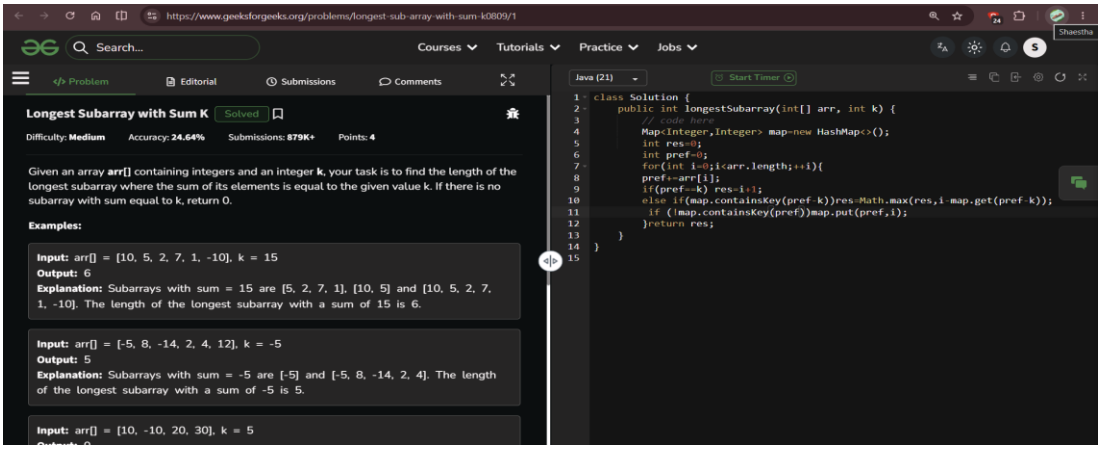
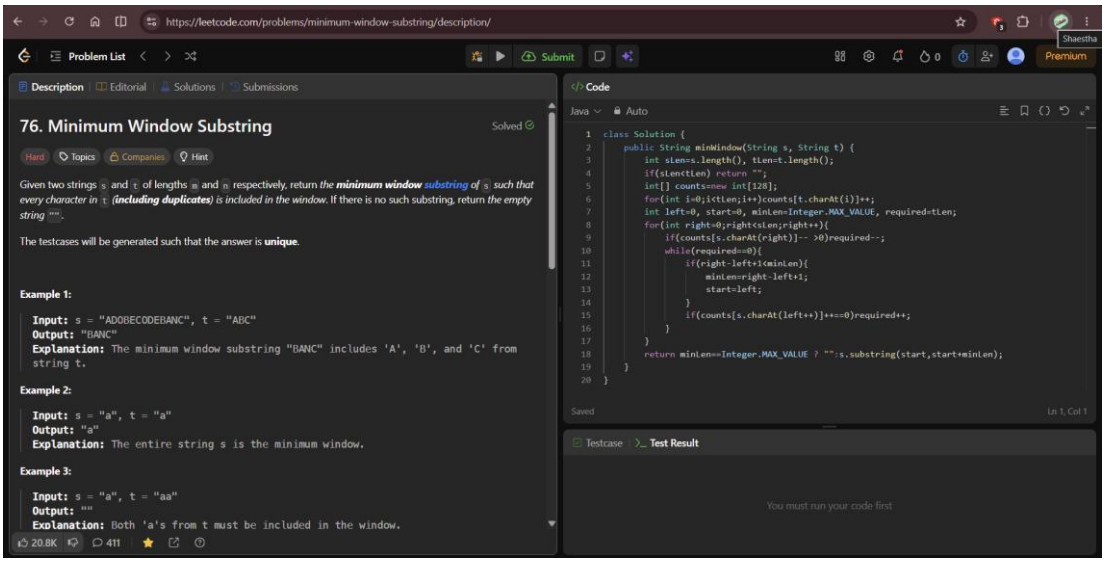
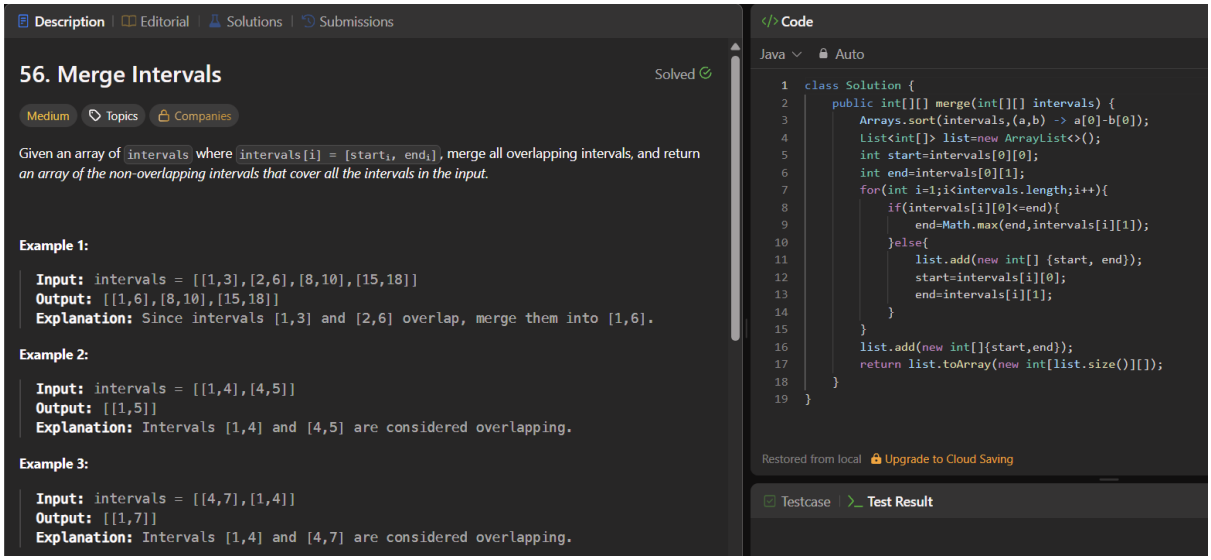
DSA Leetcode

1. 

2. 

3. 

4.

703. Kth Largest Element in a Stream

Solved

Easy | Topics | Companies

You are part of a university admissions office and need to keep track of the `kth` highest test score from applicants in real-time. This helps to determine cut-off marks for interviews and admissions dynamically as new applicants submit their scores.

You are tasked to implement a class which, for a given integer `k`, maintains a stream of test scores and continuously returns the `k`th highest test score **after** a new score has been submitted. More specifically, we are looking for the `k`th highest score in the sorted list of all scores.

Implement the `KthLargest` class:

- `KthLargest(int k, int[] nums)` Initializes the object with the integer `k` and the stream of test scores `nums`.
- `int add(int val)` Adds a new test score `val` to the stream and returns the element representing the `k`th largest element in the pool of test scores so far.

Example 1:

Java | Auto

```java
class KthLargest {
    private PriorityQueue<Integer> minHeap;
    private int k;
    public KthLargest(int k, int[] nums) {
        this.k=k;
        this.minHeap=new PriorityQueue<>();
        for(int num:nums)add(num);
    }
    public int add(int val) {
        minHeap.offer(val);
        if(minHeap.size()>k)minHeap.poll();
        return minHeap.peek();
    }
}

/**
 * Your KthLargest object will be instantiated and called as such:
 * KthLargest obj = new KthLargest(k, nums);
 * int param_1 = obj.add(val);
 */
```

Saved

5.

Problem List | Submit

Description | Editorial | Solutions | Submissions

142. Linked List Cycle II

Solved

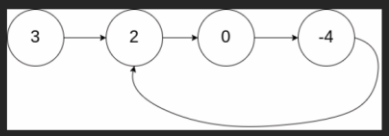
Medium | Topics | Companies

Given the `head` of a linked list, return *the node where the cycle begins. If there is no cycle, return* `null`.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that tail's `next` pointer is connected to (**0-indexed**). It is `-1` if there is no cycle. **Note that** `pos` **is not passed as a parameter**.

Do not modify the linked list.

Example 1:



Code

Java | Auto

```java
 *     }
 * }
 */
public class Solution {
    public ListNode detectCycle(ListNode head) {
        ListNode slow =head, fast=head;
        while(fast!=null && fast.next!=null){
            slow = slow.next;
            fast = fast.next.next;
            if(slow==fast){
                slow = head;
                while(slow!=fast){
                    slow = slow.next;
                    fast = fast.next;
                } return slow;
            }
        } return null;
    }
}
```

Saved

Testcase | Test Result

6.

Problem List | Submit

Description | Editorial | Solutions | Submissions

236. Lowest Common Ancestor of a Binary Tree

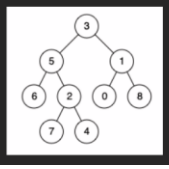
Solved

Medium | Topics | Companies

Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree.

According to the definition of LCA on Wikipedia: "The lowest common ancestor is defined between two nodes `p` and `q` as the lowest node in `T` that has both `p` and `q` as descendants (where we allow **a node to be a descendant of itself**)."

Example 1:



Code

Java | Auto

```java
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        return lca(root,p,q);
    }
    TreeNode lca(TreeNode root, TreeNode p,TreeNode q){
        if(root==null) return null;
        if(root==p || root==q) return root;
        TreeNode left=lca(root.left,p,q);
        TreeNode right=lca(root.right,p,q);
        if(left!=null && right!=null) return root;
        if(left!=null) return left;
        if(right!=null) return right;
        return null;
    }
}
```

Saved

Testcase | Test Result

7. Description | Editorial | Solutions | Submissions

1091. Shortest Path in Binary Matrix

Solved

Medium | Topics | Companies | Hint

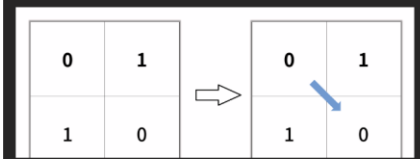
Given an `n x n` binary matrix `grid`, return *the length of the shortest* ***clear path*** *in the matrix*. If there is no clear path, return `-1`.

A **clear path** in a binary matrix is a path from the **top-left** cell (i.e., `(0, 0)`) to the **bottom-right** cell (i.e., `(n - 1, n - 1)`) such that:

- All the visited cells of the path are `0`.
- All the adjacent cells of the path are **8-directionally** connected (i.e., they are different and they share an edge or a corner).

The **length of a clear path** is the number of visited cells of this path.

Example 1:



Code

Java | Auto

```java
class Solution {
    public int shortestPathBinaryMatrix(int[][] grid) {
        int n = grid.length;
        if (grid[0][0] != 0 || grid[n-1][n-1] != 0) return -1;
        boolean[][] visit = new boolean[n][n];
        int[][] child = {
            {0,1},{0,-1},{1,0},{-1,0},
            {1,1},{1,-1},{-1,1},{-1,-1}
        };
        Queue<int[]> q = new LinkedList<>();
        q.offer(new int[]{0,0});
        visit[0][0] = true;
        int levels = 1;
        while (!q.isEmpty()) {
            int l = q.size();
            levels = levels + 1;
            for (int k = 0; k < l; k++) {
                int[] t = q.poll();
                int i = t[0], j = t[1];
                if (i == n-1 && j == n-1) return levels - 1;
                for (int d = 0; d < 8; d++) {
                    int ni = i + child[d][0];
                    int nj = j + child[d][1];
                    if (ni >= 0 && nj >= 0 && ni < n && nj < n &&
                        !visit[ni][nj] && grid[ni][nj] == 0) {
                        visit[ni][nj] = true;
                        q.offer(new int[]{ni, nj});
                    }
```

Saved

8.

Problem List | Submit

Description | Editorial | Solutions | Submissions

210. Course Schedule II

Solved

Medium | Topics | Companies | Hint

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [a_i, b_i]` indicates that you **must** take course `b_i` first if you want to take course `a_i`.

- For example, the pair `[0, 1]`, indicates that to take course `0` you have to first take course `1`.

Return *the ordering of courses you should take to finish all courses*. If there are many valid answers, return **any** of them. If it is impossible to finish all courses, return **an empty array**.

Example 1:

```
Input: numCourses = 2, prerequisites = [[1,0]]
Output: [0,1]
Explanation: There are a total of 2 courses to take. To take course 1 you should
have finished course 0. So the correct course order is [0,1].
```

Example 2:

```
Input: numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]
Output: [0,2,1,3]
Explanation: There are a total of 4 courses to take. To take course 3 you should
have finished both courses 1 and 2. Both courses 1 and 2 should be taken after you
finished course 0.
So one correct course order is [0,1,2,3]. Another correct ordering is [0,2,1,3].
```

Example 3:

Code

Java | Auto

```java
public int[] findOrder(int numCourses, int[][] prerequisites) {
    int[] res = new int[numCourses];
    int k=0;
    List<List<Integer>> graph = new ArrayList<>();
    for (int i = 0; i < numCourses; i++) {
        graph.add(new ArrayList<>());
    }
    int[] indegree = new int[numCourses];
    for (int[] pre : prerequisites) {
        int course = pre[0];
        int prereq = pre[1];
        graph.get(prereq).add(course);
        indegree[course]++;
    }
    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0) {
            queue.add(i);
        }
    }
    int count = 0;
    while (!queue.isEmpty()) {
        int current = queue.poll();
        res[k++]=current;
        for (int neighbor : graph.get(current)) {
            indegree[neighbor]--;
            if (indegree[neighbor] == 0) {
                queue.add(neighbor);
            }
        }
    }
    if(k!=numCourses) return new int[]{};
    return res;
```

Saved

.9

Problem List | Submit

Description | Editorial | Solutions | Submissions

322. Coin Change

Solved

Medium | Topics | Companies

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return the **fewest** number of coins that you need to make up that amount. If that amount of money cannot be made up by any combination of the coins, return -1.

You may assume that you have an infinite number of each kind of coin.

Example 1:

```
Input: coins = [1,2,5], amount = 11
Output: 3
Explanation: 11 = 5 + 5 + 1
```

Example 2:

```
Input: coins = [2], amount = 3
Output: -1
```

Example 3:

Code

Java | Auto

```java
class Solution {
    public int coinChange(int[] coins, int amount) {
        int n=coins.length;
        int[][] dp=new int[n][amount+1];
        int max = amount + 1;
        for(int i = 0; i <= amount; i++){
            if(i%coins[0] == 0) {
                dp[0][i] = i / coins[0];
            }else{
                dp[0][i] = max;
            }
        }
        for(int i = 1; i < n; i++){
            for(int j = 0; j <= amount; j++){
                int notPick = dp[i-1][j];
                int pick = max;
                if(coins[i] <= j){
                    pick = 1 + dp[i][j - coins[i]];
                }
                dp[i][j] = Math.min(pick, notPick);
            }
        }
        return dp[n-1][amount] == max ? -1 : dp[n-1][amount];
    }
}
```